

PYTHON EXCEPTION HANDLING

Six real-world engineering scenarios — analytics, data ingestion, config loading, service startup and banking — each with stubbed code and marked blanks where you write the `try / except / finally` logic.

```
handling_pattern.py

try:
    result = risky_operation(payload)
except SpecificError as err:
    log.warning("handled: %s", err)
    result = fallback()
else:
    commit(result)
finally:
    release_resources()
```

PROBLEMS

6 Tasks

LEVEL

Beginner

FOCUS

Built-in + Custom

EXCEPTIONS IN SCOPE

ZeroDivisionError

OverflowError

IndexError

KeyError

ValueError

TypeError

FileNotFoundError

OSError

ImportError

RuntimeError

try / except / finally

raise

Custom exceptions

The exception hierarchy you'll work with

Every exception below descends from `BaseException`. Catching a parent (e.g. `LookupError`) also catches its children (`IndexError`, `KeyError`) — useful when one handler should cover a whole family. Catch the most specific type that fits the failure.



Find the blanks

Every **# TODO** line (highlighted) marks where your handling goes. The risky statement is already written.

Catch precisely

Prefer the narrowest exception that fits. Reach for a parent only when you deliberately want to cover the whole family.

Fail gracefully

A handler should report clearly and let the program continue — never swallow an error silently with a bare `except: pass`.

#	SCENARIO	EXCEPTIONS PRACTISED
P1	Sales analytics	ArithmeticError • ZeroDivisionError • OverflowError
P2	Sales leaderboard	LookupError • IndexError • KeyError
P3	CSV ingestion	ValueError • TypeError
P4	Config loader	FileNotFoundError • OSError/IOError • finally
P5	Service startup	ImportError • RuntimeError
P6	Banking transactions	Custom exceptions • raise • KeyError

P1

Sales Analytics — Reporting Metrics

ArithmeticError

ZeroDivisionError

OverflowError

A reporting job computes average order value and projects future revenue. Two operations fail on edge data: a period with **zero orders**, and a projection compounded over so many periods the float **overflows**.

YOUR TASK

1. Guard the division so a period with 0 orders returns 0.0 instead of crashing.
2. Handle OverflowError in the projection, with a general ArithmeticError safety net.

```
report_metrics.py PYTHON
1 def average_order_value(total_revenue, num_orders):
2     """Return the average revenue per order for a reporting period."""
3     # TODO: total_revenue / num_orders raises ZeroDivisionError when a
4     #       period has 0 orders. Handle it and return 0.0 instead of crashing.
5     avg = total_revenue / num_orders
6     return round(avg, 2)
7
8
9 def project_revenue(current_revenue, growth_rate, periods):
10    """Project revenue compounding at 'growth_rate' over N periods."""
11    # TODO: with a very large 'periods', the result raises OverflowError.
12    #       Handle OverflowError, and add a general ArithmeticError safety net.
13    projected = current_revenue * (1.0 + growth_rate) ** periods
14    return round(projected, 2)
15
16
17 # --- Test cases (do not change) ---
18 print(average_order_value(15000, 120))    # 125.0
19 print(average_order_value(15000, 0))      # must NOT crash → 0.0
20 print(project_revenue(50000, 0.08, 5))    # normal projection
21 print(project_revenue(1e6, 8.0, 100000))  # must NOT crash (overflow)
```

P2

Sales Leaderboard Lookup

LookupError

IndexError

KeyError

You query a sales team two ways: by **rank** (list index) and by **rep name** (dict key). A bad rank and an unknown rep raise two different lookup errors — plus a bonus that catches both in one handler.

YOUR TASK

1. Handle `IndexError` for an out-of-range rank; return `None`.
2. Handle `KeyError` for a rep with no record; return `None`.
3. Bonus: guard BOTH lookups with a single `except LookupError` block.

```
leaderboard.py PYTHON

1 # Leaderboard ordered by rank; quota attainment keyed by rep name.
2 top_reps = ["A. Chen", "R. Patel", "M. Silva", "K. Osei"] # index 0 = rank 1
3 quota_hit = {"A. Chen": 112, "R. Patel": 98, "M. Silva": 87} # % of quota
4
5 def rep_at_rank(rank):
6     """Return the sales rep at a given 1-based rank."""
7     # TODO: top_reps[rank - 1] raises IndexError for an out-of-range rank.
8     #     Handle it: "No rep at rank {rank}." and return None.
9     return top_reps[rank - 1]
10
11
12 def quota_for(rep):
13     """Return the quota-attainment % for a rep."""
14     # TODO: quota_hit[rep] raises KeyError if the rep has no record.
15     #     Handle it: "No quota record for {rep}." and return None.
16     return quota_hit[rep]
17
18
19 def safe_report(rank, rep):
20     """Bonus: use ONE 'except LookupError' block to guard BOTH
21         top_reps[rank - 1] and quota_hit[rep], then print both results."""
22     # TODO: single try / except LookupError wrapping both lookups.
23     pass
24
25
26 # --- Test cases ---
27 print(rep_at_rank(2)) # R. Patel
28 print(rep_at_rank(10)) # handled, no crash
29 print(quota_for("M. Silva")) # 87
30 print(quota_for("J. Doe")) # handled, no crash
```

P3

CSV Data Ingestion

ValueError

TypeError

An import job reads raw text fields from a CSV. Converting non-numeric text raises **ValueError**; summing a column that still contains a stray string raises **TypeError**.

YOUR TASK

1. Handle `ValueError` in `parse_amount`; return `None` for invalid text.
2. Handle `TypeError` in `column_total`; report the non-numeric value.

```
csv_ingest.py PYTHON
1 def parse_amount(raw):
2     """Convert a raw text field (e.g. from a CSV) into a float amount."""
3     # TODO: float(raw) raises ValueError for non-numeric text like 'N/A'.
4     #     Handle ValueError and return None for invalid input.
5     return float(raw)
6
7
8 def column_total(values):
9     """Sum a numeric column. A stray string entry raises TypeError."""
10    # TODO: sum(values) raises TypeError if the column mixes numbers and text.
11    #     Handle TypeError and report that the column has a non-numeric value.
12    return sum(values)
13
14
15 # --- Test cases ---
16 print(parse_amount("1999.50"))    # 1999.5
17 print(parse_amount("N/A"))        # handled → None
18 print(column_total([100, 250, 75])) # 425
19 print(column_total([100, "250", 75])) # handled, no crash
```

P4

Application Config Loader

FileNotFoundError

OSError / IOError

finally

A service loads settings from a config file at startup. Practise the full shape: handle the missing file, handle any other I/O failure, and use **finally** for cleanup that always runs.

YOUR TASK

1. Handle `FileNotFoundError` when the file is missing.
2. Handle `OSError` (`IOError` is an alias) for other I/O failures.
3. Add a `finally` block that always prints a completion message.

```
config_loader.py PYTHON
1 def load_config(path):
2     """Read and print each line of an application config file."""
3     # TODO:
4     # 1) open(path) raises FileNotFoundError if the file is missing --
5     #    handle it with a clear message.
6     # 2) Reading can raise OSError (IOError is an alias for it) -- handle it too.
7     # 3) Use 'finally' to ALWAYS print: "config load attempt finished".
8     f = open(path, "r")
9     for line in f:
10        print(line.strip())
11    f.close()
12
13
14 # --- Test cases ---
15 load_config("app.config")      # may or may not exist
16 load_config("does_not_exist.cfg") # must be handled gracefully
```

P5

Service Startup — Optional Dependency & State

ImportError

RuntimeError

A service prefers a faster optional library but must degrade gracefully when it isn't installed, and it must refuse to run a query before the database connection is established.

YOUR TASK

1. Handle ImportError for the optional library; fall back to the standard json module.
2. Wrap run_query() in except RuntimeError so a bad connection state doesn't crash.

```
service_startup.py PYTHON

1 def get_json_parser():
2     """Use the faster 'orjson' library if installed, else fall back."""
3     # TODO: 'import orjson' raises ImportError (ModuleNotFoundError) when it
4     #     isn't installed. Handle it, print "orjson not available -- falling
5     #     back to standard json", and import the built-in json instead.
6     import orjson
7     return orjson
8
9
10 class ReportService:
11     def __init__(self):
12         self.connected = False # connection not opened yet
13     def run_query(self):
14         if not self.connected:
15             raise RuntimeError("Database connection not established.")
16         return "query results"
17
18
19 def generate_report(service):
20     """TODO: call service.run_query() inside try/except RuntimeError,
21         print the error message, and keep the program running."""
22     pass
23
24
25 # --- Test cases ---
26 get_json_parser() # handled → standard json fallback
27 generate_report(ReportService()) # RuntimeError handled, no crash
```

P6

Banking Transactions — Custom Exceptions

CAPSTONE

Custom exceptions

KeyError

raise

The capstone in three steps: **define** two domain-specific exceptions, **raise** the right one inside `withdraw()`, and **handle** all three (including a built-in `KeyError`) in `process_withdrawal()`.

YOUR TASK

1. STEP 1 — define `InsufficientFundsError` and `InvalidAmountError` (the latter inheriting from `ValueError`).
2. STEP 2 — raise the correct exception for each failure case inside `withdraw()`.
3. STEP 3 — catch and message all three cases in `process_withdrawal()`.

```
banking.py PYTHON
1 accounts = {"ACC-1001": 5000.0, "ACC-1002": 250.0, "ACC-1003": 0.0}
2
3 # TODO STEP 1 -- Define two custom exceptions:
4 # - InsufficientFundsError : raised when a withdrawal exceeds the balance
5 # - InvalidAmountError    : raised when amount ≤ 0
6 #   (make InvalidAmountError inherit from ValueError)
7 #
8 # class InsufficientFundsError(____):
9 #     pass
10 # class InvalidAmountError(____):
11 #     pass
12
13
14 def withdraw(account_id, amount):
15     """Withdraw 'amount' from an account and return the new balance."""
16 # TODO STEP 2 -- Raise the right exception:
17 # - account_id not in accounts → raise KeyError(account_id)
18 # - amount ≤ 0                → raise InvalidAmountError
19 # - amount > accounts[account_id] → raise InsufficientFundsError
20 # Otherwise subtract the amount, update the balance, and return it.
21     pass
22
23
24 def process_withdrawal(account_id, amount):
25     """TODO STEP 3 -- call withdraw(...) inside try/except and handle:
26     - KeyError → "Unknown account: {account_id}"
27     - InvalidAmountError → "Withdrawal amount must be positive."
28     - InsufficientFundsError → "Insufficient funds in {account_id}."
29     """
30     pass
31
32
33 # --- Test cases ---
34 process_withdrawal("ACC-1001", 1200) # OK → 3800.0
35 process_withdrawal("ACC-9999", 100)  # KeyError handled
36 process_withdrawal("ACC-1002", -50)  # InvalidAmountError handled
37 process_withdrawal("ACC-1003", 100)  # InsufficientFundsError handled
```

Six scenarios, one discipline.



You've guarded arithmetic, lookups, conversions, file and import failures, invalid runtime state, and written your own exception types.

The same pattern scales to any codebase: anticipate the failure, catch precisely, recover or report.

- ▶ Run each file before and after your fix — confirm the crash, then confirm it's gone.
- ▶ Reproduce every commented test case; match the described behaviour exactly.
- ▶ Ask of each handler: is this the *narrowest* exception that fits?
- ▶ Avoid `except Exception: pass` — always report what happened.
- ▶ In P6, notice how inheriting from `ValueError` lets a caller catch either the specific or the general type.